Lecturer: Yetmwork Tesfaye

MSc. in Computer Science and Engineering

College of Computing and Informatics

Department of Computer Science

Chapter 7: Consistency and Replication

Objectives

At the end of this Chapter Students be able to know:-

- Introduction to Consistency
- Reasons for Replication
- Replication as Scaling Technique
- Consistency Models
- Data-Centric Consistency Models

What is Replication?

- An important issue in distributed systems is the replication of data.
- Make multiple copies of a data object and ensure that all copies are identical.
- Keeping local copy of data in multiple location
- It is the continuous copying of data changes from one database (publisher) to another database (subscriber).
- The two databases are generally located on a different physical servers, resulting in a load balancing.

Reasons for Replication(why Replication?)

- **Reliability**
  - If one replica is unavailable or crashes, use another
  - Protect against corrupted data
- **Performance**
  - Scale with size of the distributed system (replicated web servers)
  - Scale in geographically distributed systems (web proxies)

Consistency Models

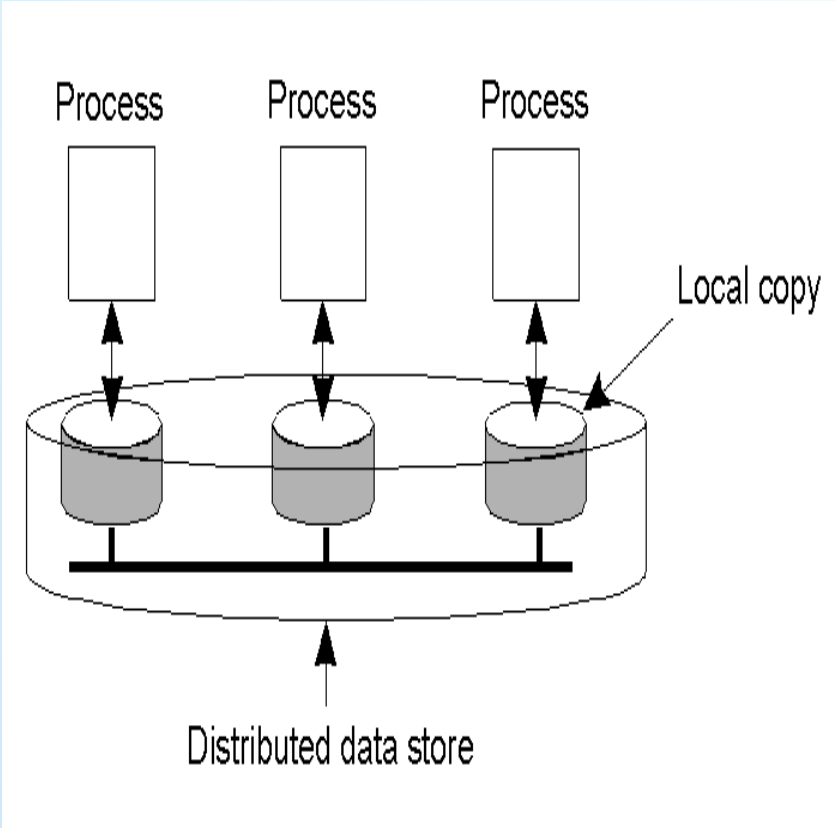
- Key issue: need to maintain *consistency* of replicated data
- *Consistency* each replica node has same view of data at a given time
- Each read request get most recent value of write
  - If one copy is modified, others become inconsistent
- To keep replicas consistent, we generally need to ensure that all **conflicting** operations are done in the same order everywhere.
- **Conflicting operations:** From the world of transactions:
  - **Read-write conflict**: a read operation and a write operation act concurrently
  - **Write-write conflict**: two concurrent write operations

Consistency Models(contd.)

- A consistency model is essentially a contract between processes and the data store.
- It says that if processes agree to obey certain rules, the store promises to work correctly.
- Each model effectively restricts the values that a read operation on a data item can return.

Data-Centric Consistency Models

- Each process that can access data has its own local copy
- Write operations are propagated to the other copies
- All models attempt to return the results of the last write for a read operation



Data-Centric Consistency Models

- Strict consistency
- Sequential Consistency
- Causal consistency
- FIFO consistency
- Weak consistency
- Release consistency
- Entry consistency

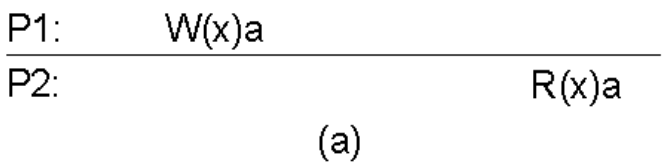
Strict Consistency

- We draw the operations of a process along a time axis.
- The time axis is always drawn horizontally, with time increasing from left to right.
- We use the notation Wi (x)a to denote that process Pi writes value a to data item x. Similarly, Ri (x)b represents the fact that process Pi reads x and is returned the value b.
- We assume that each data item has initial value NIL.

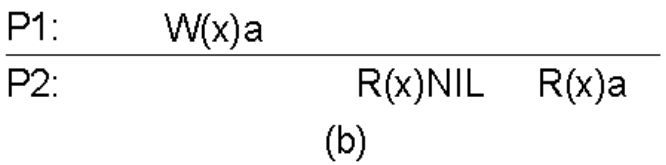
Strict Consistency

Any read on a data item x returns a value corresponding to the result of the most recent write on x

two operations in the same time interval are said to conflict if they operate on the same data and one of them is a write operation



(a)



(b)

- Behavior of two processes, operating on the same data item.

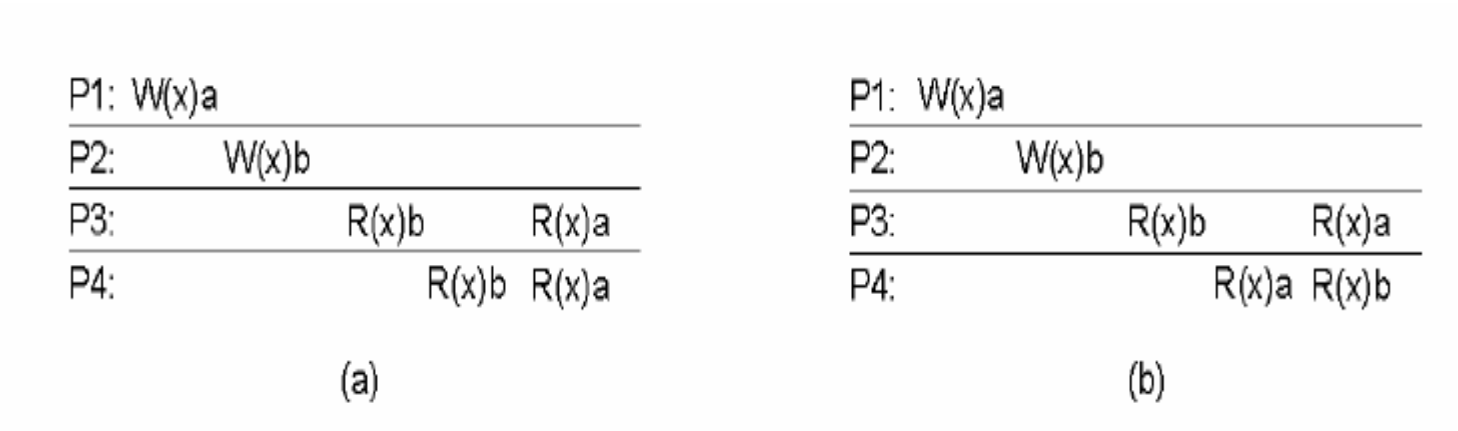
a) A strictly consistent store.

b) A store that is not strictly consistent.

Sequential Consistency

- Any **valid interleaving of read and write** operations is acceptable and all processes see the same interleaving of operations
- The result of any execution is the same as if the (read and write) operations by all processes on the data store were executed in some sequential order and the operations of each individual process appear in this sequence in the order specified by its program.
- Nothing about time
- Weaker than strict consistency

Sequential Consistency



A) sequentially consistent data-store

- P3 and P4 sees the same interleaving of write operations by P1 and P2 (see P2 first and then P1)

B) A data-store that is not sequentially consistent

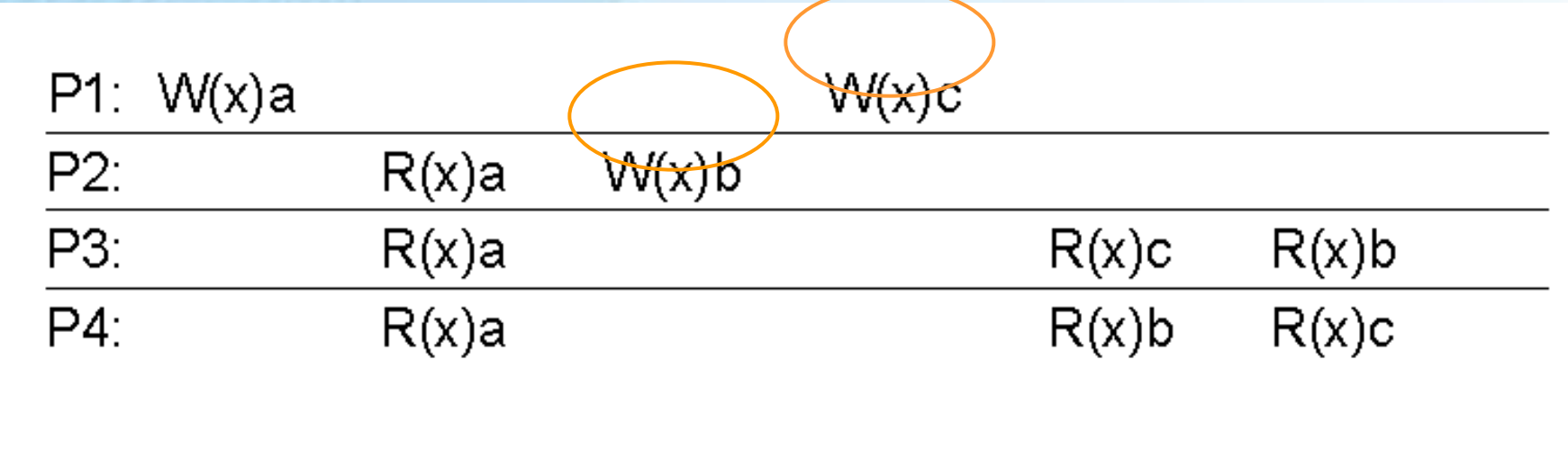
- P3 and P4 do not see the same interleaving of write operations by P1 and P2

Causal Consistency(1)

- When there is a read followed by a write, the two events are potentially causally related.
- Operation not causally related are said concurrent.
- Necessary condition:

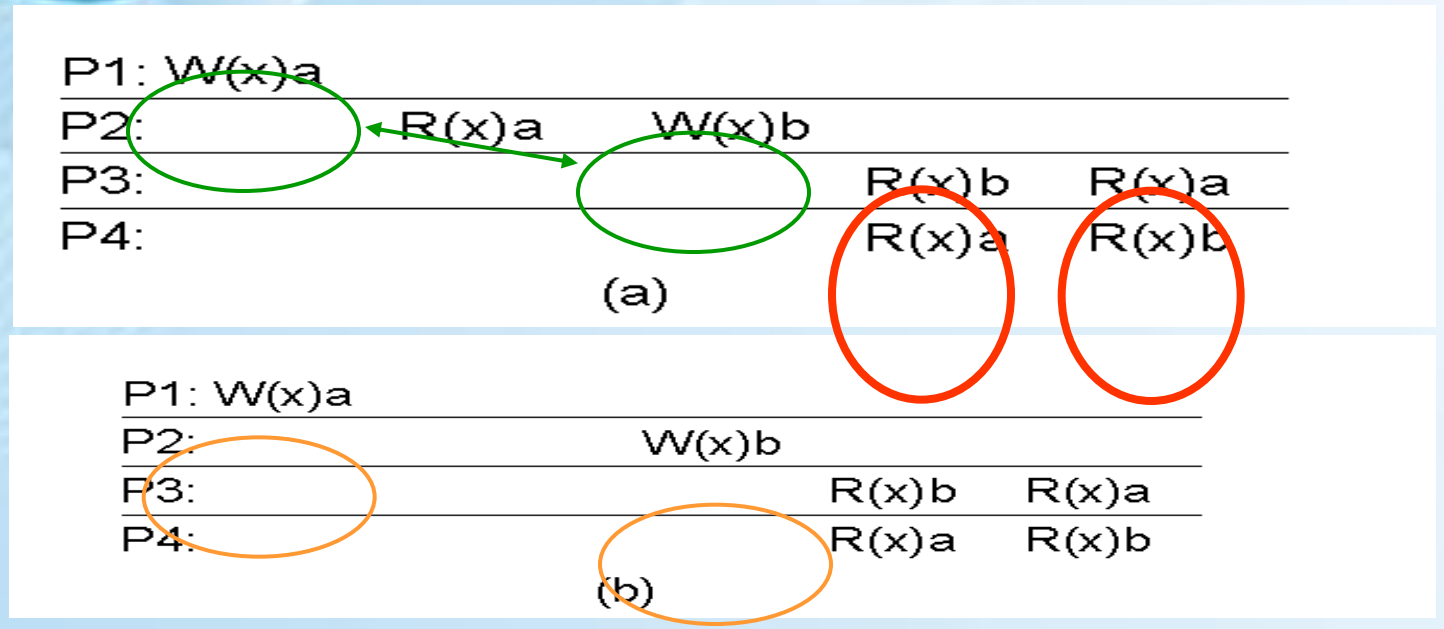
  Writes that are potentially causally related must be seen by all processes in the same order. Concurrent writes may be seen in a different order on different machines.

Causal Consistency (2)



- This sequence is allowed with a causally-consistent store, but not with sequentially or strictly consistent store.
- Note that the writes $W_2(x)b$ and $W_1(x)c$ are concurrent
- Causal consistency requires keeping tracks of which processes have seen which writes.

Causal Consistency (3)



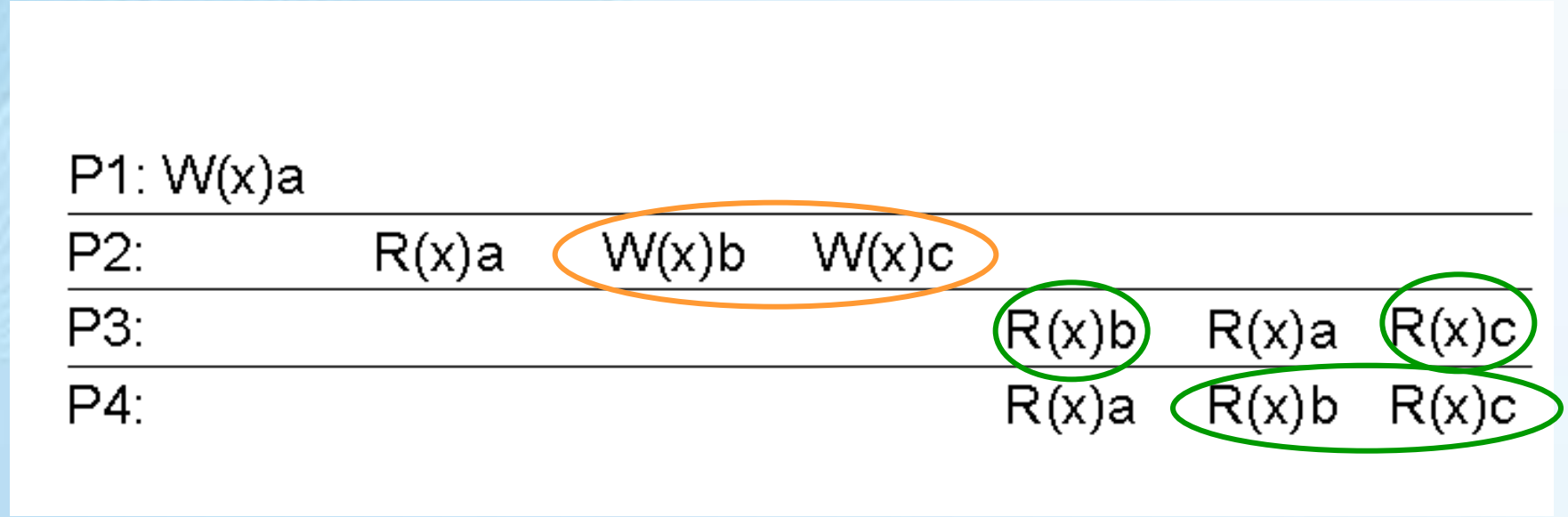
a) Violation of causal-consistency – P2's write is related to P1's write due to the read on 'x' giving 'a' (all processes must see them in the same order).

b) A causally-consistent data-store: the read has been removed, so the two writes are now *concurrent*. The reads by P3 and P4 are now OK.

FIFO Consistency (1)

- **Relaxing consistency requirements we drop causality**
- **Necessary Condition:**
  Writes done by a single process are seen by all other processes in the order in which they were issued, but writes from different processes may be seen in a different order by different processes.
- All writes generated by different processes are considered concurrent
- It is easy to implement

FIFO Consistency (2)



- A valid sequence of events of FIFO consistency. It is not valid for causal consistency

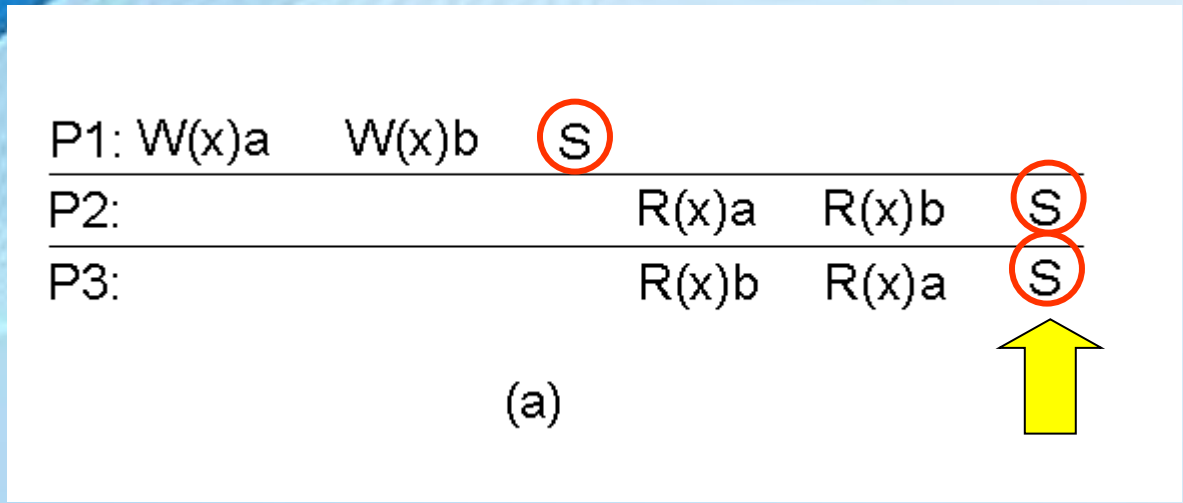
Weak Consistency

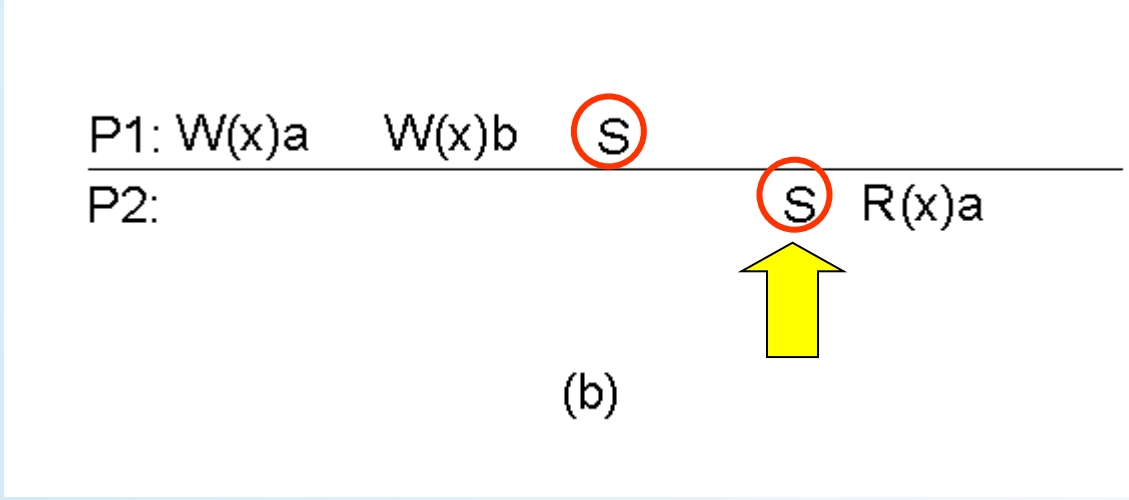
- We can release the requirements of writes within the same process seen in order everywhere introducing a synchronization variable.
- A synchronization operation synchronize all local copies of the data store.
- Properties of weak consistency:
  - No operation on a synchronization variable is allowed to be performed until all previous writes have been completed everywhere.
  - No read or write operation on data items are allowed to be performed until all previous operations to synchronization variables have been performed.
- It forces consistency on a **group** of operations, not on individual write and read

Weak Consistency

- A synchronize(S) operation by P, causes all writes by P to be propagated to all other replicas and all external writes are propagated to P.
  - Process P forces the just written value out to all the other replicas
  - Process P can be sure it’s getting the most recently written value before it reads.

Weak Consistency(Example)





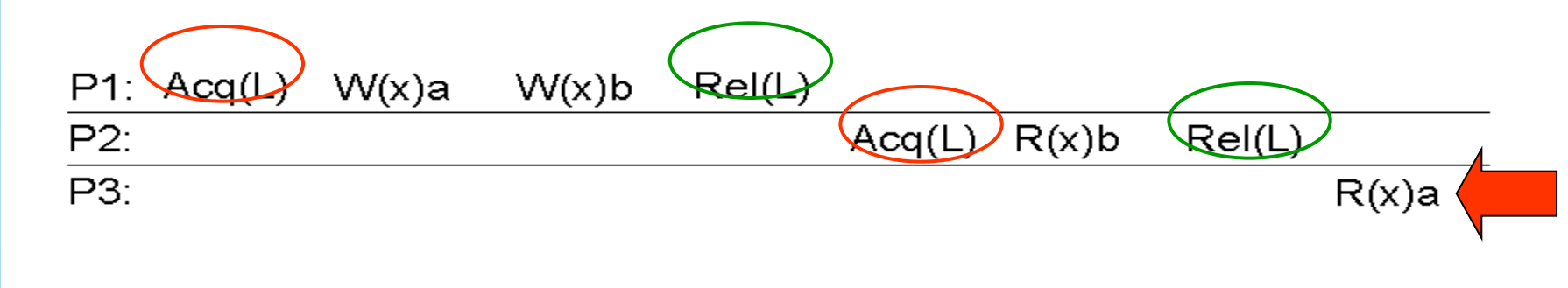
a. A valid sequence of events for weak consistency. This is because P2 and P3 have yet to synchronize, so there's no guarantees about the value in 'x'.

b. An invalid sequence for weak consistency. P2 has synchronized, so it cannot read 'a' from 'x' – it should be getting 'b'.

Release Consistency (1)

- If it is possible to know the difference between entering a critical region or leaving it, a more efficient implementation might be possible. To do that, two kinds of synchronization variables are needed.

  Release consistency : acquire operation to tell that a critical region is being entered; release operation when a critical region is to be exited.



No consistency guarantee

- Process P3 has not performed an acquire, so there are no guarantees that the read of x is consistent.

Release Consistency(2)

Rules:

- Before a read or write operation on shared data is performed, all previous acquires done by the process must have completed successfully.
- Before a release is allowed to be performed, all previous reads and writes by the process must have completed.
- Explicit acquire and release calls are required.

Entry Consistency

- each shared data item associated with a synchronization variables.
- In order to access consistent data, each synchronization variable must be explicitly acquired.
- Release consistency affects all shared data but entry consistency affects only those shared data associated with a synchronization variable.
- Acquire and release are still used, and the data store meets the following conditions:

Conditions:

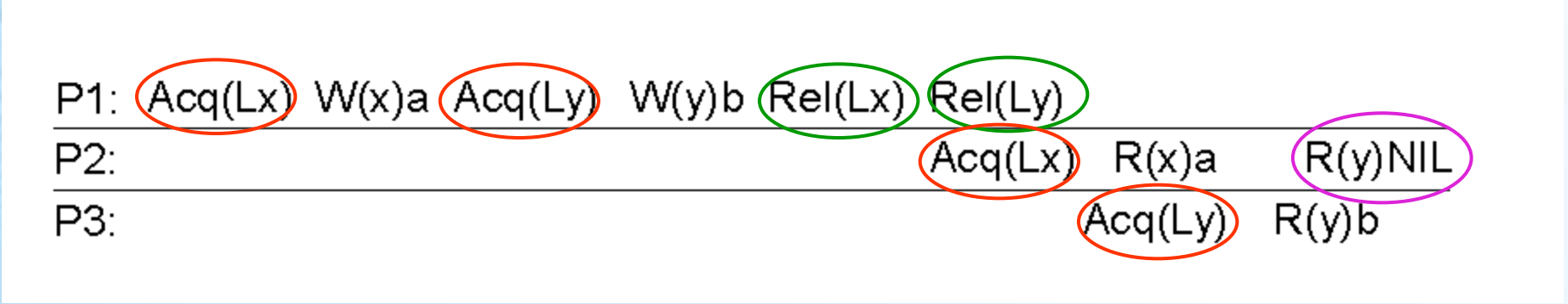
- An acquire access of a synchronization variable is not allowed to perform with respect to a process until all updates to the guarded shared data have been performed with respect to that process.

Entry Consistency

Conditions:

- Before an exclusive mode access to a synchronization variable by a process is allowed to perform with respect to that process, no other process may hold the synchronization variable, not even in nonexclusive mode.
- After an exclusive mode access to a synchronization variable has been performed, any other process's next nonexclusive mode access to that synchronization variable may not be performed until it has performed with respect to that variable's owner.

Entry Consistency (Example)



- A valid event sequence for entry consistency. Lock are associated with each data item

Thank You!!